

Lab exercises

Part 10 — AI Architecture · 10-ai-architecture/lab/EXERCISES.md

Lab — AI Architecture

Work top to bottom. Each exercise has a stated deliverable; if you can't produce it, you haven't finished.

1. Draw your own layered architecture (1h)

Take a real (or realistic) AI feature you know well — at work, or one you've used as a product. Draw the full layered architecture from notes/01 §2 [10-ai-architecture/lab/notes/01-reference-architecture.md] as it actually exists (or as you believe it exists) for that feature: client → gateway → orchestration → model layer → tool/retrieval → data → observability.

Deliverable: the diagram, plus one paragraph on which layer you're LEAST confident about and why. **Check:** if you can't name what lives in the orchestration layer specifically, that's the gap to close first.

2. Extend the model gateway (2h)

Using code/model_gateway_simulator.py as a base, add: - A fourth simulated provider with a different cost/latency/failure profile. - A "sticky routing" mode: once a request is routed to a provider, retries for that request stay on the same provider unless it's the one that failed. - A report comparing sticky vs. non-sticky retry routing on total cost and success rate.

Check: sticky routing should reduce cache-miss-style waste in some scenario you construct — show the numbers, don't just assert it.

3. Break your own semantic cache (1.5h)

Using code/semantic_cache_correctness.py: - **3a.** Find (or construct) a THIRD pair of near-duplicate queries that require different answers, distinct from the one in the script. Show the cache failing on it at the script's default threshold. - **3b.** Tighten the threshold until your new pair is handled correctly. Report what happened to the hit rate on the genuine paraphrase pairs. - **3c.** Implement the "entity/number guard" mentioned in notes/01 §4.1 [10-ai-architecture/lab/notes/01-reference-architecture.md] — a cheap check that blocks a cache hit when the two queries contain different numbers or named entities — and show it fixes your case WITHOUT hurting the genuine-paraphrase hit rate.

Deliverable: a before/after table: hit rate and error rate, threshold-only vs. threshold+guard.

4. Design a circuit breaker policy (1h, no code required)

For a hypothetical service calling three providers (a frontier model, a mid-tier model, and a self-hosted open model), specify: - The failure threshold that trips the breaker for each (should they be the same? why or why not?). - The cooldown/half-open retry policy. - What happens to in-flight requests when a breaker trips mid-request. - What your on-call engineer sees when this happens — is it a page, a dashboard color, or silence?

Deliverable: a one-page policy doc. **Check:** could someone else implement your policy from this doc alone, without asking you a clarifying question?

5. Trace a request through the security path (1h)

Using notes/02 §3 [10-ai-architecture/lab/notes/02-production-patterns.md]'s request-flow diagram, take a single user prompt containing an injection attempt (borrow one from ../13-ai-security/ if you've done that module, or construct your own) and annotate, at each layer of your Exercise 1 diagram, what control (if any) should catch it, and what happens if that layer is skipped.

Deliverable: the annotated diagram. **Check:** if every layer says "the model will handle it," you've found the actual gap this exercise exists to expose.

6. The capstone architecture, critiqued (2h)

Read notes/02 §6 [10-ai-architecture/lab/notes/02-production-patterns.md]'s capstone reference architecture closely. Pick ONE component and argue, in writing, for a different choice — e.g. "I would use a message queue instead of direct synchronous calls here because..." Be specific about what you gain and what you give up.

Deliverable: a half-page technical argument. **Check:** does it name a concrete trade-off (latency, cost, operational complexity) rather than just a preference?